

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

CHINTHAN G UPADHYA (1BM24CS084)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **CHINTHAN G UPADHYA (1BM24CS084)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Mayanka Gupta
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4-8
2	Implement Johnson Trotter algorithm to generate permutations.	9-12
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	13-15
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	16-18
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	19-21
6	Implement 0/1 Knapsack problem using dynamic programming.	22-25
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	26-29
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	30-35
9	Implement Fractional Knapsack using Greedy technique.	36-38
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	39-41
11	Implement "N-Queens Problem" using Backtracking.	42-44

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph

CODE

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    printf("Enter number of vertices: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        fprintf(stderr, "Invalid number of vertices.\n");
        return 1;
    }
    int **adj = malloc(n * sizeof(int *));
    if (!adj) {
        fprintf(stderr, "Memory allocation failed.\n");
        return 1;
    }
    for (int i = 0; i < n; i++) {
        adj[i] = calloc(n, sizeof(int));
        if (!adj[i]) {
            fprintf(stderr, "Memory allocation failed.\n");
            return 1;
        }
    }
    printf("Enter adjacency matrix (0 or 1) row by row:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (scanf("%d", &adj[i][j]) != 1) {
                fprintf(stderr, "Invalid adjacency matrix input.\n");
                return 1;
            }
        }
    }
    int *indegree = calloc(n, sizeof(int));
    int *check = calloc(n, sizeof(int));
    int *topo = malloc(n * sizeof(int));
    if (!indegree || !check || !topo) {
        fprintf(stderr, "Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (adj[i][j] != 0) {
                indegree[j]++;
            }
        }
    }
}
```

```

    }
}
}int count = 0;

while (1) {
    int w = -1;
    for (int i = 0; i < n; i++) {
        if (check[i] == 0 && indegree[i] == 0) {
            w = i;
            break;
        }
    }
    if (w == -1) {
        break;
    }
    topo[count++] = w + 1;
    check[w] = 1;

    for (int v = 0; v < n; v++) {
        if (adj[w][v] != 0) {
            indegree[v]--;
        }
    }
}
if (count < n) {
    printf("No Topological Sequence\n");
} else {
    printf("Topological Sequence: ");
    for (int i = 0; i < count; i++) {
        printf("%d ", topo[i]);
    }
    printf("\n");
}
for (int i = 0; i < n; i++) {
    free(adj[i]);
}
free(adj);
free(indegree);
free(check);
free(topo);
return 0;
}

```

OUTPUT

```
Enter number of vertices: 4
Enter adjacency matrix (0 or 1) row by row:
0 1 1 0
0 0 0 1
0 1 0 1
0 0 0 0
Topological Sequence: 1 3 2 4
```

LEETCODE - 207

CODE

```
int dfs(int course, int** graph, int* graphSize, int* visited) {
    if (visited[course] == 1)
        return 0;
    if (visited[course] == 2)
        return 1;

    visited[course] = 1;

    for (int i = 0; i < graphSize[course]; i++) {
        if (!dfs(graph[course][i], graph, graphSize, visited))
            return 0;
    }

    visited[course] = 2;
    return 1;
}

bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
    int** graph = (int**)malloc(numCourses * sizeof(int*));
    int* graphSize = (int*)calloc(numCourses, sizeof(int));
    for (int i = 0; i < numCourses; i++)
        graph[i] = (int*)malloc(numCourses * sizeof(int));

    for (int i = 0; i < prerequisitesSize; i++) {
        int course = prerequisites[i][0];
        int pre = prerequisites[i][1];
        graph[pre][graphSize[pre]++] = course;
    }

    int* visited = (int*)calloc(numCourses, sizeof(int));

    for (int i = 0; i < numCourses; i++) {
        if (!dfs(i, graph, graphSize, visited)) {
            // Free memory before returning
            for (int j = 0; j < numCourses; j++) free(graph[j]);
            free(graph);
        }
    }
}
```

```

        free(graphSize);
        free(visited);
        return false;
    }
}

for (int i = 0; i < numCourses; i++) free(graph[i]);
free(graph);
free(graphSize);
free(visited);
return true;
}

```

OUTPUT

Accepted 54 / 54 testcases passed
Chinthan_G submitted at Jun 01, 2026 20:04

Runtime 42 ms | Beats 21.64%
Memory 110.14 MB | Beats 21.02%

12.9% of solutions used 0 ms of runtime

Code C

```

1 int dfs(int course, int** graph, int* graphSize, int* visited) {
2     if (visited[course] == 1) // currently visiting -> cycle found
3         return 0;
4     if (visited[course] == 2) // already processed -> no cycle
5         return 1;
6
7     visited[course] = 1; // mark as visiting
8
9     for (int i = 0; i < graphSize[course]; i++) {
10         if (!dfs(graph[course][i], graph, graphSize, visited))
11             return 0;
12     }
13
14     visited[course] = 2; // mark as done
15     return 1;
16 }
17
18 bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
19     int** graph = (int**)malloc(numCourses * sizeof(int*));
20     int* graphSize = (int*)calloc(numCourses, sizeof(int));
21     for (int i = 0; i < numCourses; i++)
22         graph[i] = (int*)malloc(numCourses * sizeof(int));
23
24     for (int i = 0; i < prerequisitesSize; i++) {
25         int course = prerequisites[i][0];
26         int pre = prerequisites[i][1];
27         graph[course][pre] = 1;
28         graphSize[course]++;
29     }
30
31     for (int i = 0; i < numCourses; i++)
32         if (!dfs(i, graph, graphSize, visited))
33             return false;
34
35     return true;
36 }

```

Saved Ln 1, Col 1

Testcase Test Result

Case 1 Case 2 +

</> Source

OBSERVATION

Date: _____
Page: _____

```

while (--c < n) {
    int v = q[++i];
    topo[count++] = v;
    for (int i = 0; i < n; i++) {
        if (adj[v][i] == 1) {
            in[i]--;
            if (in[i] == 0) {
                q[++i] = i;
            }
        }
    }
}

if (count == n) {
    printf("Graph has a cycle, topological\n");
    printf("ordering not possible.");
} else {
    printf("Topological ordering");
    for (int i = 0; i < n; i++) {
        printf("%d ", topo[i]);
    }
    return 0;
}

```

O/P

Enter number of vertices: 4

Enter Adjacency Matrix: 0 1 1 0
 0 0 0 1
 0 0 0 1
 0 0 0 0

0 1 2 3 → Topological ordering

Program 2:

Implement Johnson Trotter algorithm to generate permutations.

CODE

```
#include <stdio.h>
#include <stdlib.h>

#define LEFT -1
#define RIGHT 1

typedef struct {
    int value;
    int direction;
} Element;

void printPermutation(Element *arr, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i].value);
    }
    printf("\n");
}

int isMobile(Element *arr, int i, int n) {
    if (arr[i].direction == LEFT && i > 0) {
        return arr[i].value > arr[i - 1].value;
    }
    if (arr[i].direction == RIGHT && i < n - 1) {
        return arr[i].value > arr[i + 1].value;
    }
    return 0;
}

int findLargestMobileIndex(Element *arr, int n) {
    int largestMobileIndex = -1;
    for (int i = 0; i < n; i++) {
        if (isMobile(arr, i, n)) {
            if (largestMobileIndex == -1 || arr[i].value > arr[largestMobileIndex].value) {
                largestMobileIndex = i;
            }
        }
    }
    return largestMobileIndex;
}

void reverseDirectionsOfGreater(Element *arr, int n, int mobileValue) {
    for (int i = 0; i < n; i++) {
        if (arr[i].value > mobileValue) {
            arr[i].direction = (arr[i].direction == LEFT) ? RIGHT : LEFT;
        }
    }
}
```

```

int main() {
    int n;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        fprintf(stderr, "Invalid input. Please enter a positive integer.\n");
        return 1;
    }

    Element *arr = malloc(n * sizeof(Element));
    if (arr == NULL) {
        fprintf(stderr, "Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        arr[i].value = i + 1;
        arr[i].direction = LEFT;
    }

    printf("Johnson-Trotter permutations:\n");
    printPermutation(arr, n);

    while (1) {
        int mobileIndex = findLargestMobileIndex(arr, n);
        if (mobileIndex == -1) {
            break;
        }

        int swapIndex = (arr[mobileIndex].direction == LEFT) ? mobileIndex - 1 : mobileIndex + 1;

        Element temp = arr[mobileIndex];
        arr[mobileIndex] = arr[swapIndex];
        arr[swapIndex] = temp;

        reverseDirectionsOfGreater(arr, n, arr[swapIndex].value);

        printPermutation(arr, n);
    }

    free(arr);
    return 0;
}

```

OUTPUT

```
Enter number of elements: 4
Johnson-Trotter permutations:
1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3
4 1 3 2
1 4 3 2
1 3 4 2
1 3 2 4
3 1 2 4
3 1 4 2
3 4 1 2
4 3 1 2
4 3 2 1
3 4 2 1
3 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4
```

OBSERVATION

Output
For $n=3$,
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

Time Complexity
 $T(n) = nT(n-1) + O(n)$
Put $n-1$ in n
 $T(n-1) = (n-1)T(n-2) + O(n-1)$
 $T(n) = n(n-1)T(n-2) + (n+1)$
 $T(n) = n(n-1)(n-2)T(n-3) + \dots + (n-k+1)$
 $T(n) = n! + O(n!)$
 $T(n) = O(n!)$

16/3

LAB PROGRAM 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

CODE

```
#include<stdio.h>
#include <time.h>

void mergeSort(int arr[], int n);
void merge(int arr[],int a[], int al, int b[], int bl);
void main(){
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int n = 7;

    clock_t start = clock();
    mergeSort(arr,7);
    clock_t end = clock();

    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Time taken by mergeSort: %f seconds\n", time_taken);

    for(int i=0;i<n;i++){
        printf("%d ", arr[i]);
    }
    printf("\n");
}

void mergeSort(int arr[], int n){
    if(n==1) return;
    int al = n/2;
    int bl = n-al;
    int a[al];
    int b[bl];
    for(int i=0;i<al;i++){
        a[i] = arr[i];
    }
    for(int i=0;i<bl;i++){
        b[i] = arr[al+i];
    }
    mergeSort(a,al);
    mergeSort(b,bl);
    merge(arr,a,al,b,bl);
}

void merge(int arr[],int a[], int al, int b[], int bl){
    int i=0;
    int j=0;
    int k=0;
    while(i<al && j<bl){
        if(a[i]<b[j]){
            arr[k++] = a[i++];
        }
    }
```

```

    else{
        arr[k++] = b[j++];
    }
}
while(i<al){
    arr[k++] = a[i++];
}
while(j<bl){
    arr[k++] = b[j++];
}
}
}

```

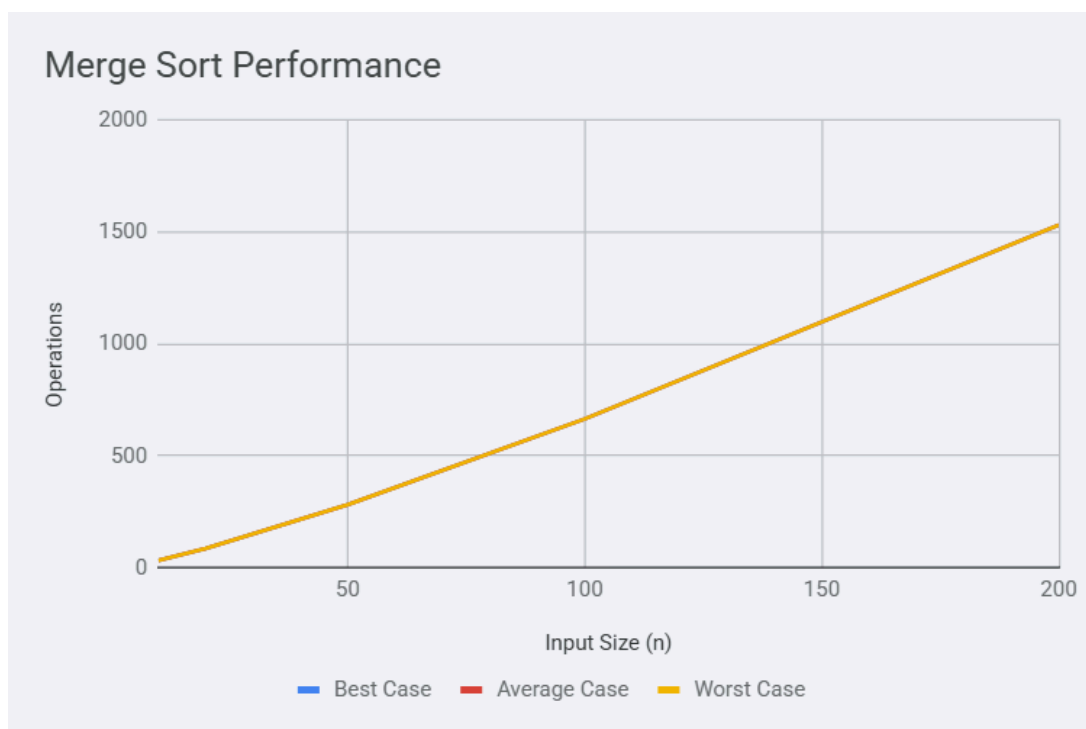
OUTPUT

```

PS C:\Users\BMSCECSE-L3-65\Desktop\1BM24CS084> cd "c:\Users\BMSCECSE-L3-65\Desktop\1BM24CS084" ; if ($?) { gcc mergesort.c -o mergesort } ; if ($?) { .\mergesort }
Time taken by mergeSort: 0.000000 seconds
3 9 10 27 38 43 82

```

GRAPH



OBSERVATION

Date: ____/____/____
Page: ____

② Merge Sort

```

#include <stdio.h>
void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    int arr[] = {8, 2, 4, 3, 9, 5, 10};
    int n = 7;
    printf("Before: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    mergeSort(arr, 0, n - 1);
    printf("After: ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    return 0;
}

```

Lab Program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

CODE

```
#include <stdio.h>
#include <time.h>

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) return 0;

    int arr[100];
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    clock_t start = clock();
    quickSort(arr, 0, n - 1);
    clock_t end = clock();
    double timeTaken = (double)(end - start) / CLOCKS_PER_SEC;
```



```

printf("Sorted array:\n");
for (int i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
printf("\n");
printf("Time taken: %.6f seconds\n", timeTaken);
return 0;
}

```

OUTPUT

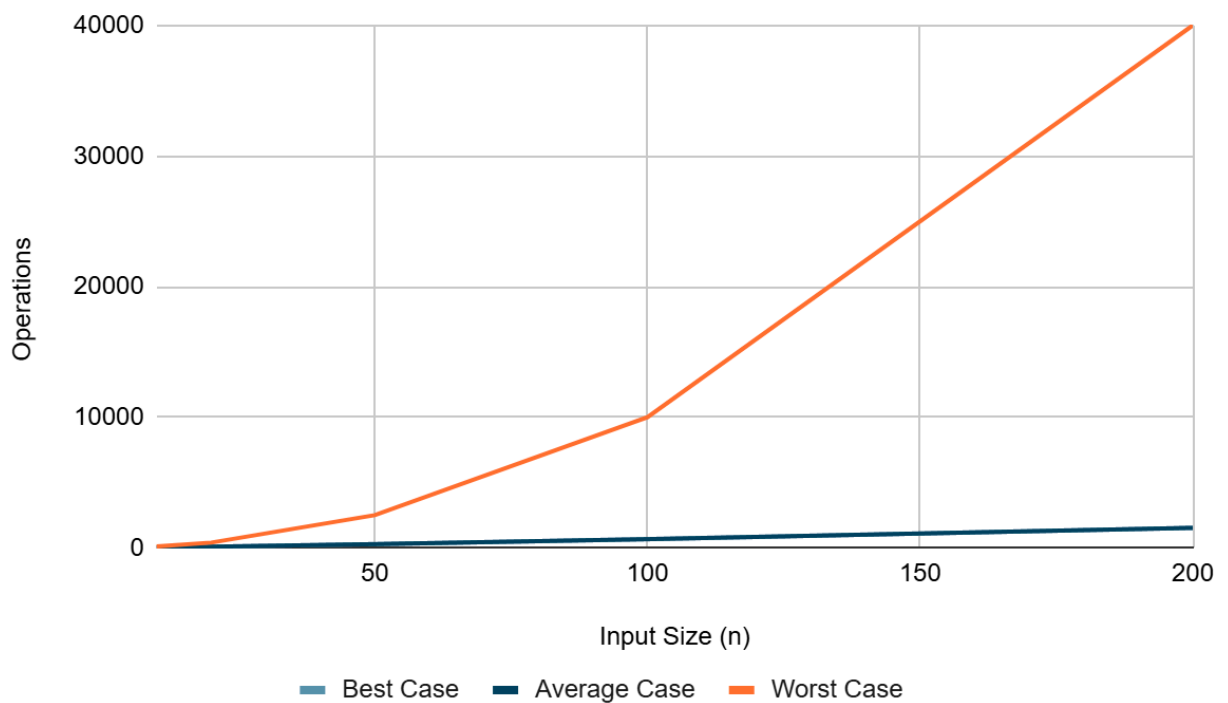
```

PS C:\Users\BMSCECSE-L3-65\Desktop\1BM24CS084> cd "c:\Users\BMSCECSE-L3-65\Desktop\1BM24CS084\" ; if ($?) { gcc quicksort.c -o quicksort } ; if ($?) { .\quicksort }
Enter number of elements: 5
Enter 5 elements:
15
14
12
19
25
Sorted array:
12 14 15 19 25
Time taken: 0.000000 seconds

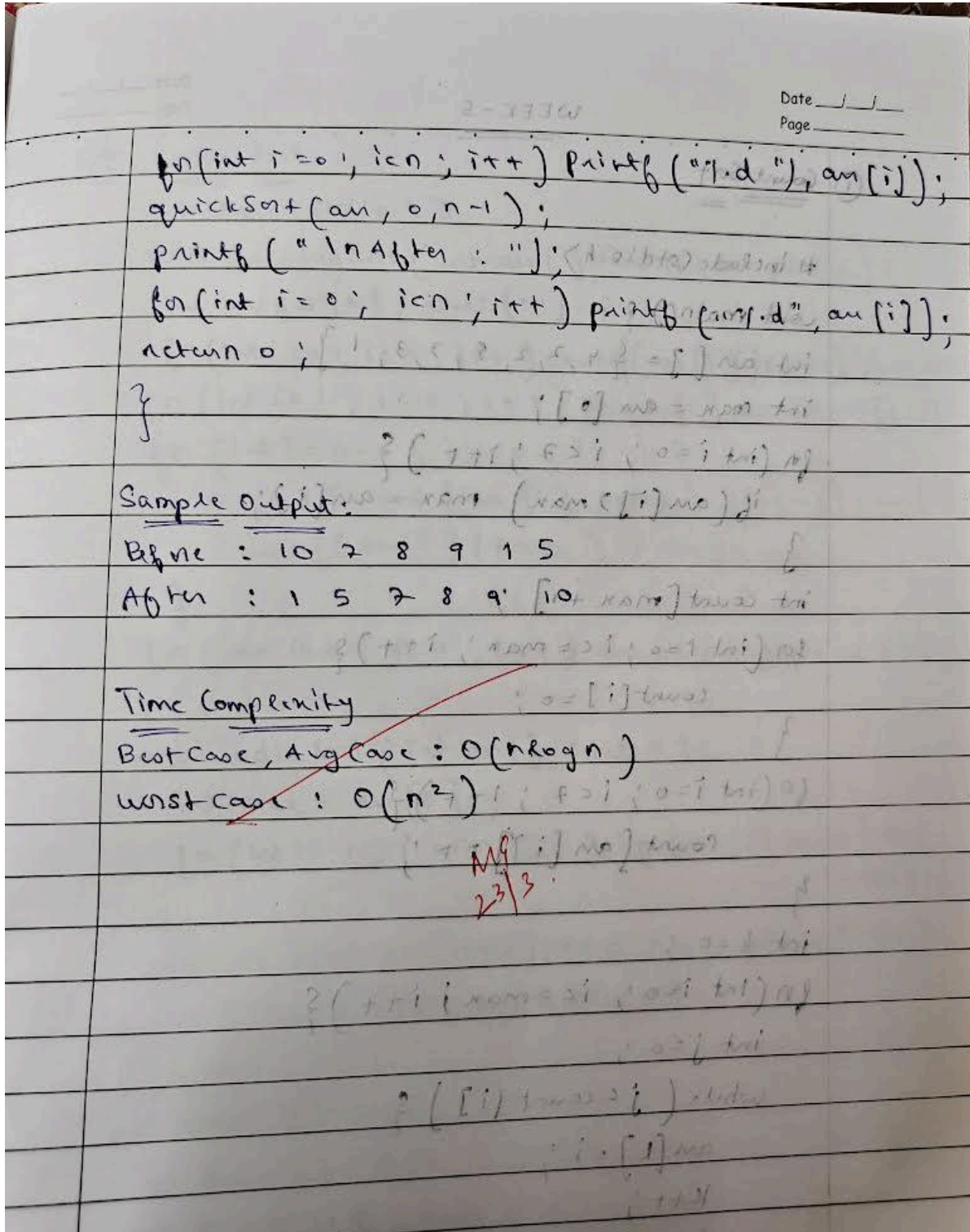
```

GRAPH

Quick Sort Performance



OBSERVATION



LAB PROGRAM 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

CODE

```
void heapify(int a[],int n,int p){
    int c, item;
    item = a[p];
    c=2*p+1;
    while(c<=n-1){
        if(c+1<=n-1){
            if(a[c]<a[c+1]){
                c++;
            }
        }
        if(item<a[c]){
            a[p] = a[c];
            p = c;
            c = 2*p+1;
        }else{
            break;
        }
    }
    a[p] = item;
}

void build_heap(int a[],int n){
    int i;
    for(i=(n-1)/2;i>=0;i--){
        heapify(a,n,i);
    }
}

void heap_sort(int a[],int n){
    int i,temp;
    build_heap(a,n);
    for(i=n-1;i>0;i--){
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;
        heapify(a,i,0);
    }
}

#include <stdio.h>
#include <time.h>
int main(){
    int n,i;
    printf("Enter the number of elements: ");
    scanf("%d",&n);
    int a[n];
    printf("Enter the elements: ");
    for(i=0;i<n;i++){
```

```

        scanf("%d",&a[i]);
    }
    clock_t start = clock();
    heap_sort(a,n);
    clock_t end = clock();
    double timeTaken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Sorted array: ");
    for(i=0;i<n;i++){
        printf("%d ",a[i]);
    }
    printf("\nTime taken: %.6f seconds\n", timeTaken);
    return 0;
}

```

OUTPUT

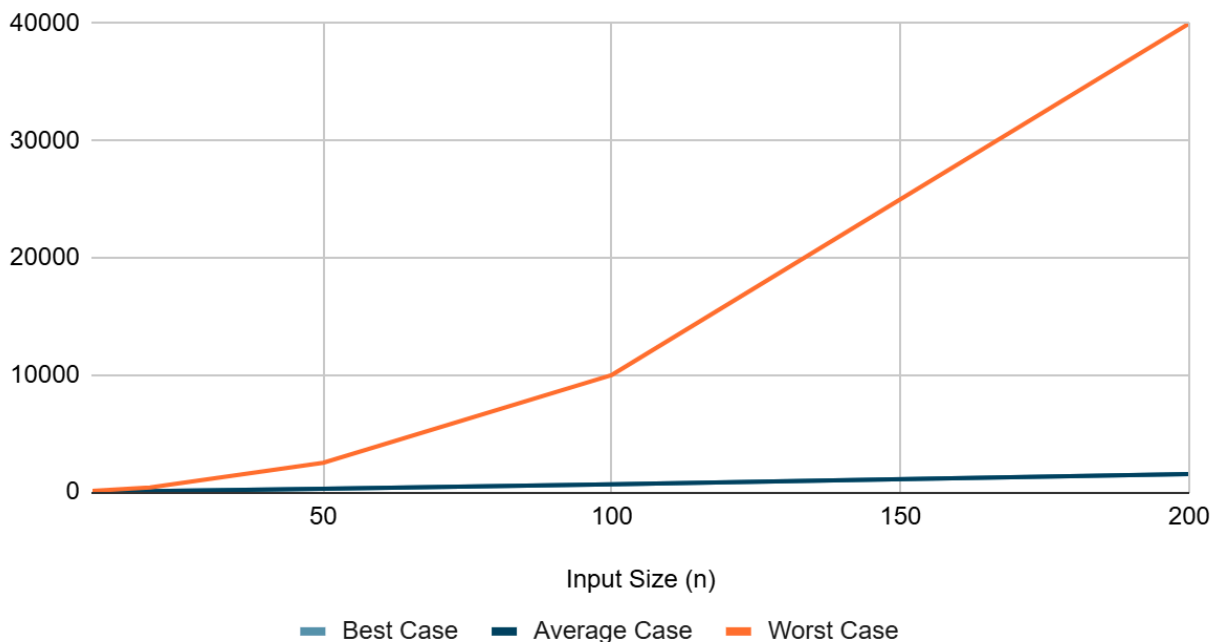
```

PS C:\Users\BMSCECSE-L3-65\Desktop\1BM24CS084> cd "c:\Users\BMSCECSE-L3-65\Desktop\1BM24CS084\" ; if ($?) { gcc heap_sort.c -o heap_sort } ; if ($?) { .\heap_sort }
Enter the number of elements: 5
Enter the elements: 15
16
12
13
25
Sorted array: 12 13 15 16 25
Time taken: 0.000000 seconds

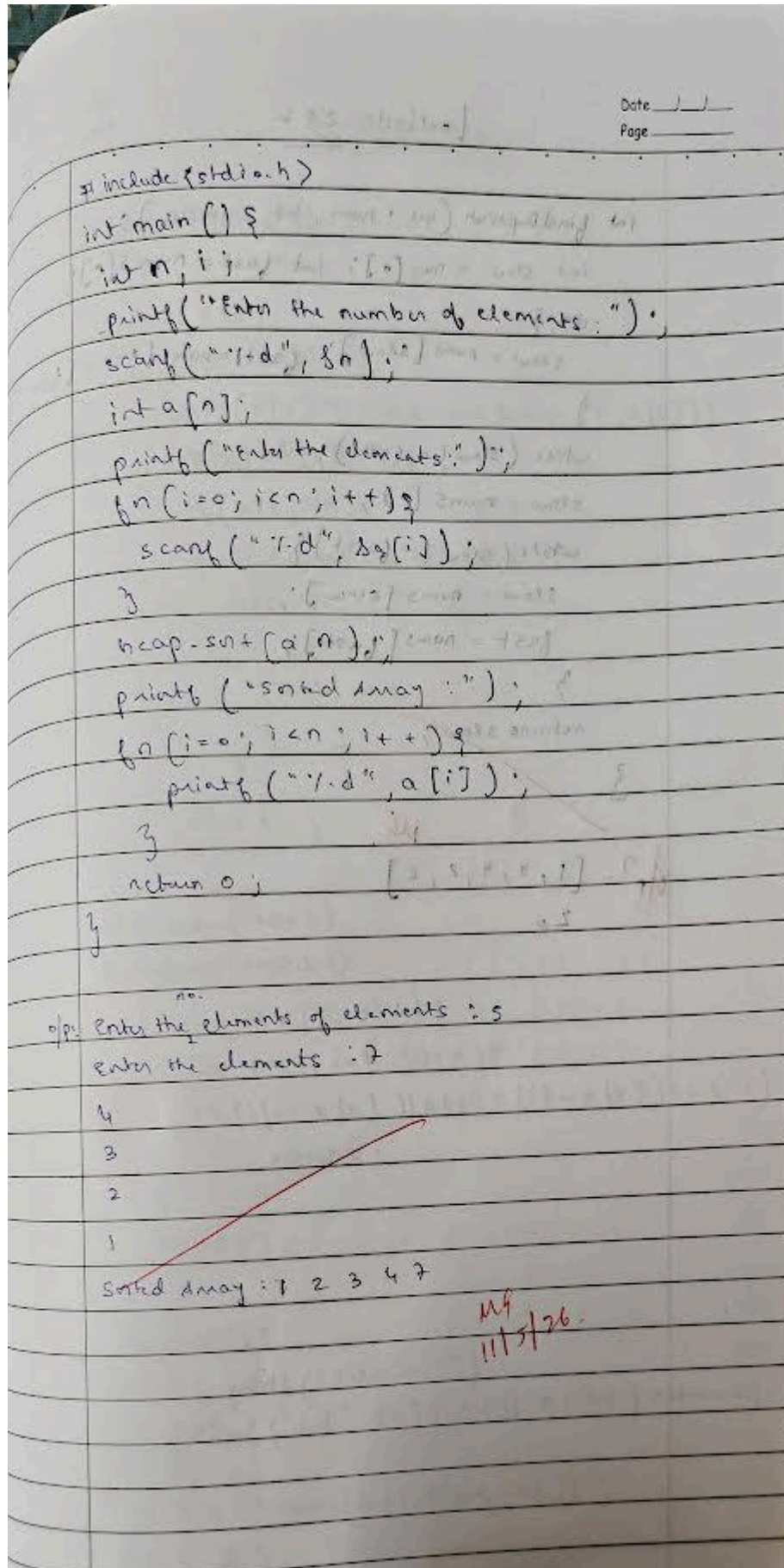
```

GRAPH

Heap Sort Performance



OBSERVATION



Lab Program 6:

Implement 0/1 Knapsack problem using dynamic programming.

CODE

```
#include <stdio.h>

int main(void) {
    float profits[3];
    int weights[3];
    float x[3] = {0.0f, 0.0f, 0.0f};
    int maxcap;
    float profit = 0.0f;

    printf("Enter profits\n");
    for (int i = 0; i < 3; i++) {
        scanf("%f", &profits[i]);
    }

    printf("Enter weights\n");
    for (int i = 0; i < 3; i++) {
        scanf("%d", &weights[i]);
    }

    printf("Enter max capacity\n");
    scanf("%d", &maxcap);

    float ratio[3];
    for (int i = 0; i < 3; i++) {
        ratio[i] = profits[i] / weights[i];
    }

    for (int i = 0; i < 2; i++) {
        for (int j = i + 1; j < 3; j++) {
            if (ratio[i] < ratio[j]) {
                float temp = ratio[i];
                ratio[i] = ratio[j];
                ratio[j] = temp;

                float tempProfit = profits[i];
                profits[i] = profits[j];
                profits[j] = tempProfit;

                int tempWeight = weights[i];
                weights[i] = weights[j];
                weights[j] = tempWeight;
            }
        }
    }

    int i = 0;
    while (i < 3 && maxcap > 0) {
```

```

    if (weights[i] <= maxcap) {
        x[i] = 1.0f;
        maxcap -= weights[i];
        profit += profits[i];
        i++;
    } else {
        break;
    }
}

if (i < 3 && maxcap > 0) {
    x[i] = (float)maxcap / weights[i];
    profit += x[i] * profits[i];
}

printf("Profit is %.2f\n", profit);
printf("Items taken are\n");
for (int k = 0; k < 3; k++) {
    printf("%.2f ", x[k]);
}
printf("\n");

return 0;
}

```

OUTPUT

```

PS C:\Users\BMSCECSE-L3-65\Desktop\1BM24CS084> cd "c:\Users\BMSCECSE-L3-65\Desktop\1BM24CS084\" ; if ($?) { gcc dynamicknapsack.c -o dynamicknapsack } ; if ($?) { .\dynamicknapsack }
Enter number of items: 3
Enter maximum capacity: 15
Enter weight and value for item 1: 5 14
Enter weight and value for item 2: 3 10
Enter weight and value for item 3: 8 4
Maximum value = 24

```

LEETCODE - 801

```
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
    int keep = 0;
    int swp = 1;

    for (int i = 1; i < nums1Size; i++) {
        int nKeep = INT_MAX;
        int nSwap = INT_MAX;

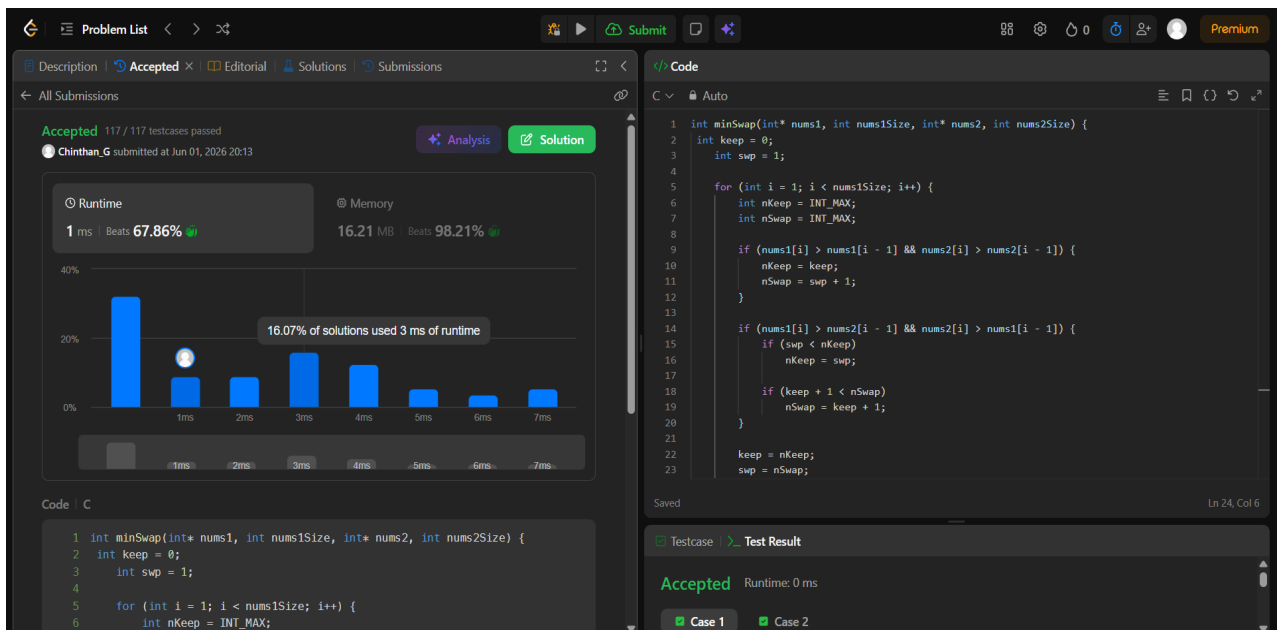
        if (nums1[i] > nums1[i - 1] && nums2[i] > nums2[i - 1]) {
            nKeep = keep;
            nSwap = swp + 1;
        }

        if (nums1[i] > nums2[i - 1] && nums2[i] > nums1[i - 1]) {
            if (swp < nKeep)
                nKeep = swp;

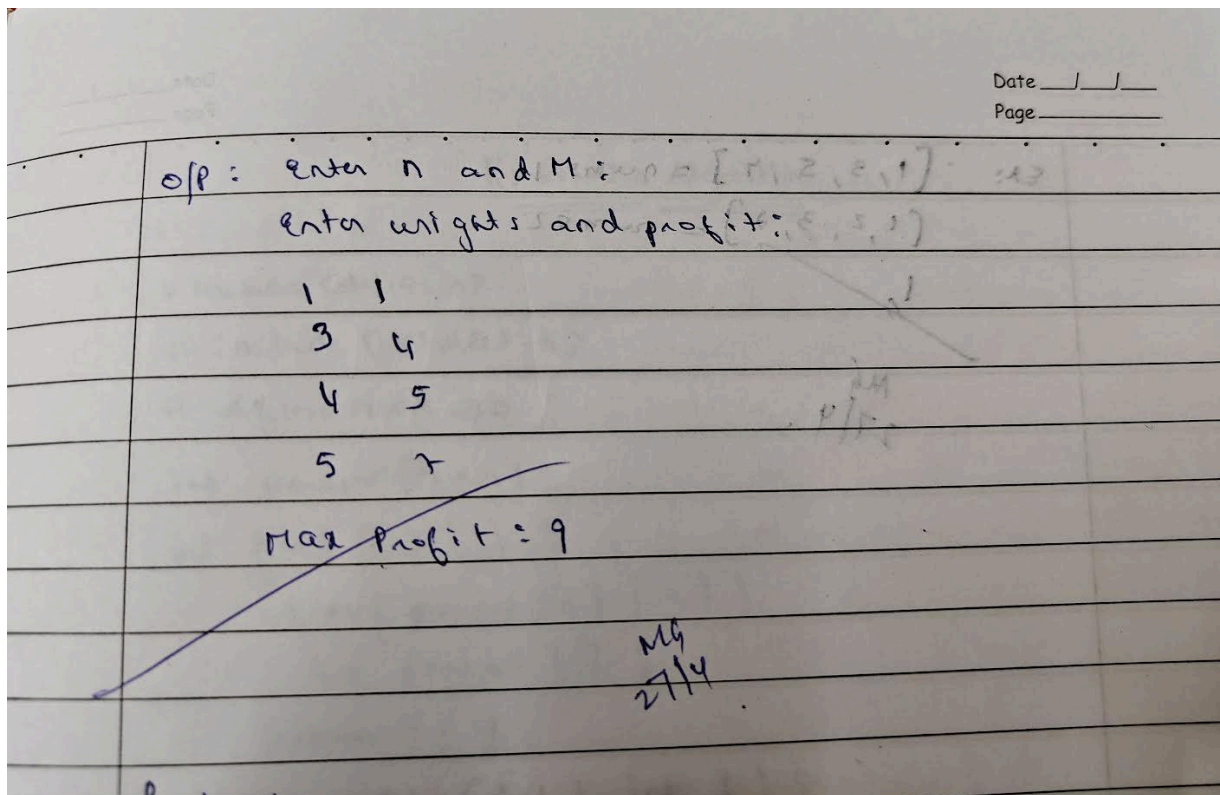
            if (keep + 1 < nSwap)
                nSwap = keep + 1;
        }

        keep = nKeep;
        swp = nSwap;
    }
    return keep < swp ? keep : swp;
}
```

OUTPUT



OBSERVATION



Lab Program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

CODE

```
#include <stdio.h>
#include <stdlib.h>

#define INF 99999

void printSolution(int **dist, int n) {
    printf("Shortest distances between every pair of vertices:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", dist[i][j]);
        }
        printf("\n");
    }
}

void floydWarshall(int **dist, int n) {
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] < INF && dist[k][j] < INF && dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
}

int main() {
    int n;
    printf("Enter number of vertices: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        return 0;
    }

    int **dist = malloc(n * sizeof(int *));
    for (int i = 0; i < n; i++) {
        dist[i] = malloc(n * sizeof(int));
    }

    printf("Enter adjacency matrix values (use %d for no edge):\n", INF);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &dist[i][j]);
        }
    }
}
```

```

    }
}

floydWarshall(dist, n);
printSolution(dist, n);

for (int i = 0; i < n; i++) {
    free(dist[i]);
}
free(dist);
return 0;
}

```

OUTPUT

```

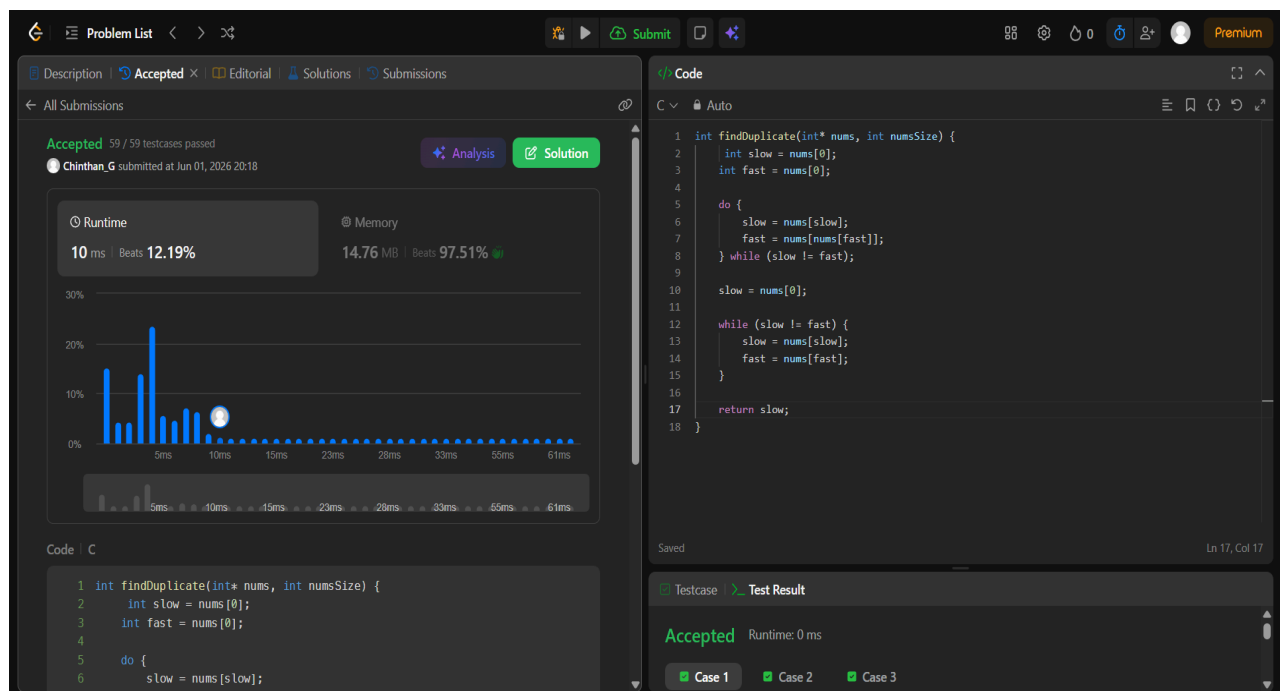
Enter number of vertices: 4
Enter adjacency matrix values (use 99999 for no edge):
0 10 99999 30
10 0 50 99999
99999 50 0 20
30 99999 20 0
Shortest distances between every pair of vertices:
0 10 50 30
10 0 50 40
50 50 0 20
30 40 20 0

```

LEETCODE - 287

```
int findDuplicate(int* nums, int numsSize) {
    int slow = nums[0];
    int fast = nums[0];
    do {
        slow = nums[slow];
        fast = nums[nums[fast]];
    } while (slow != fast);
    slow = nums[0];
    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }
    return slow;
}
```

OUTPUT



OBSERVATION

Date: / /
Page:

```

print Solution(dist); // if
}
int main() {
    int graph[V][V] = {
        {0, 5, INF, 10}, {0, 4, 99, 5, 99},
        {INF, 0, 3, INF}, {99, 0, 1, 99, 6},
        {INF, INF, 0, 1}, {2, 99, 0, 3, 99},
        {INF, INF, INF, 0}, {99, 99, 1, 0, 2},
        {2, 1, 99, 99, 4, 0}
    };
    FloydWarshall(graph);
    return 0;
}

```

O/p: Shortest Path:

0	4	5	5	2
3	0	1	4	6
2	6	0	3	5
3	3	1	0	2
1	3	5	4	0

Lab Program 8:

- (a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.
- (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

(a) Prim's Algorithm**CODE**

```
#include <stdio.h>

#define MAX 10

int minKey(int key[], int mstSet[], int V) {
    int min = 99999, min_index = -1; // use a large number instead of INT_MAX
    for (int v = 0; v < V; v++) {
        if (!mstSet[v] && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

void primMST(int graph[MAX][MAX], int V) {
    int parent[MAX], key[MAX], mstSet[MAX];

    for (int i = 0; i < V; i++) {
        key[i] = 99999;
        mstSet[i] = 0;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet, V);
        mstSet[u] = 1;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    printf("Edge \tWeight\n");
    for (int i = 1; i < V; i++)
        printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
}
```

```

int main() {
    int V, graph[MAX][MAX];
    printf("Enter number of vertices (max %d): ", MAX);
    scanf("%d", &V);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            scanf("%d", &graph[i][j]);

    primMST(graph, V);
    return 0;
}

```

OUTPUT

```

Enter number of vertices (max 10): 5
Enter adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0
Edge    Weight
0 - 1    2
1 - 2    3
0 - 3    6
1 - 4    5

```


OBSERVATION

Date / /
Page

```

printf("Edge's weight\n");
for(int i=1; i<V; i++) {
    printf("%d - %d : %d\n",
           parent[i], i, graph[i][parent[i]]);
}

int main() {
    int V, graph[100][100];
    printf("Enter number of vertices (max %d) : ", 100);
    scanf("%d", &V);

    printf("Enter adjacency matrix:\n");
    for(int i=0; i<V; i++) {
        for(int j=0; j<V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    primsT(graph, V);
    return 0;
}
    
```

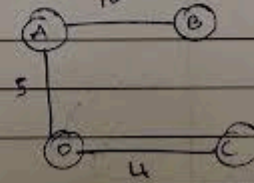
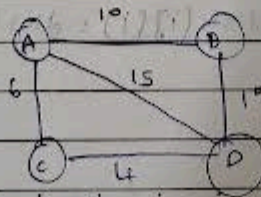
O/p: Enter number of vertices: 4
 Enter adjacency matrix

0	10	6	5
10	0	0	15
6	0	0	4
5	15	4	0

Edge weight

0 - 1 : 10
 3 - 2 : 4
 0 - 3 : 5

Minimum Cost of MST = 19



(b) Kruskal's algorithm

CODE

```
#include <stdio.h>

#define MAX_EDGES 20
#define MAX_VERTICES 10

typedef struct {
    int u, v, w;
} Edge;

void sortEdges(Edge edges[], int E) {
    for (int i = 0; i < E - 1; i++) {
        for (int j = 0; j < E - i - 1; j++) {
            if (edges[j].w > edges[j + 1].w) {
                Edge temp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = temp;
            }
        }
    }
}

int findSet(int parent[], int x) {
    while (parent[x] != x)
        x = parent[x];
    return x;
}

void unionSet(int parent[], int x, int y) {
    parent[y] = x;
}

int main() {
    int V, E;
    Edge edges[MAX_EDGES];
    int parent[MAX_VERTICES];

    printf("Enter number of vertices (max %d): ", MAX_VERTICES);
    scanf("%d", &V);
    printf("Enter number of edges (max %d): ", MAX_EDGES);
    scanf("%d", &E);

    for (int i = 0; i < E; i++) {
        printf("Enter edge %d (u v w): ", i + 1);
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    sortEdges(edges, E);

    for (int i = 0; i < V; i++)
```

```

    parent[i] = i;

printf("Edges in MST:\n");
int count = 0, totalWeight = 0;

for (int i = 0; i < E && count < V - 1; i++) {
    int u = edges[i].u;
    int v = edges[i].v;
    int setU = findSet(parent, u);
    int setV = findSet(parent, v);
    if (setU != setV) {
        printf("%d - %d : %d\n", u, v, edges[i].w);
        totalWeight += edges[i].w;
        unionSet(parent, setU, setV);
        count++;
    }
}

printf("Total weight = %d\n", totalWeight);
return 0;
}

```

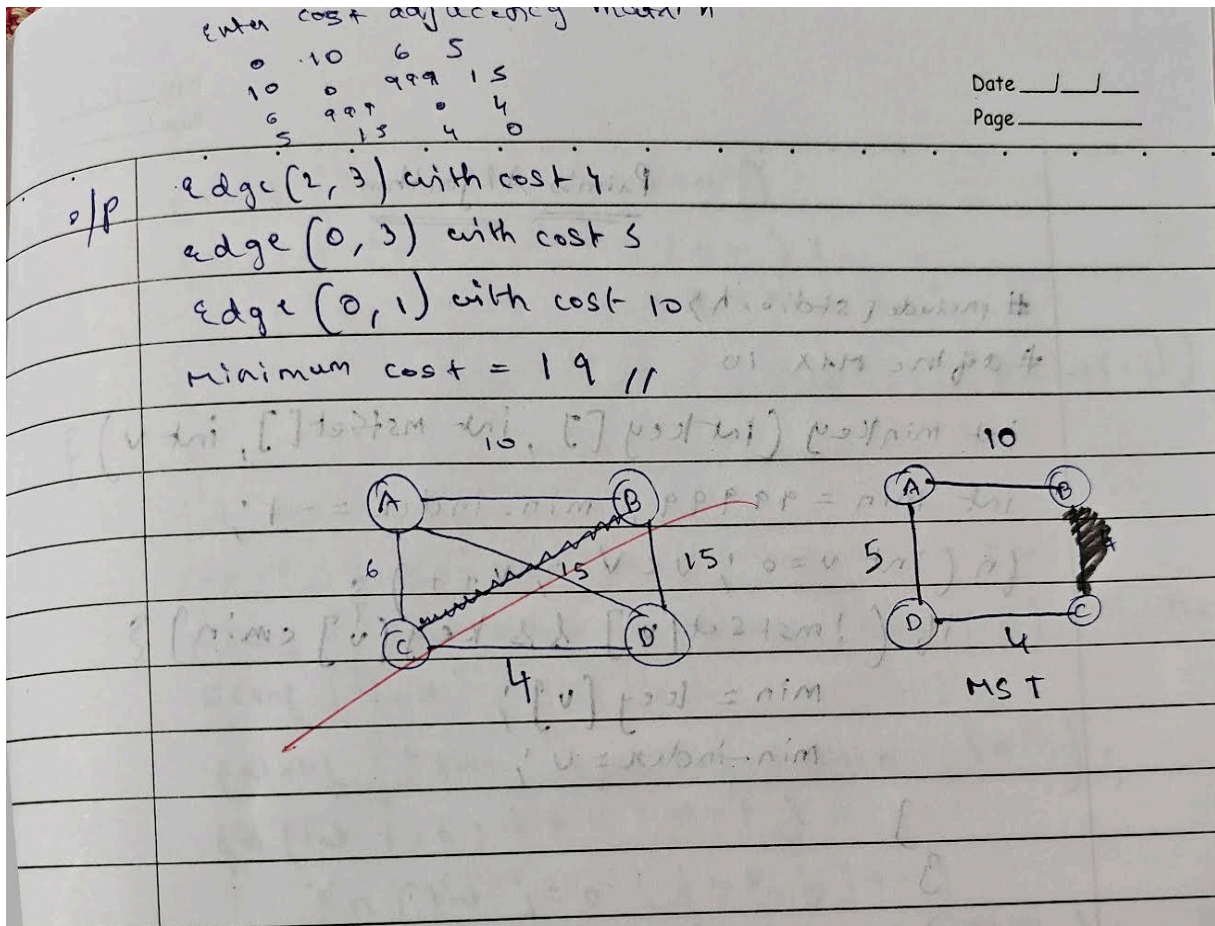
OUTPUT

```

Enter number of vertices (max 10): 4
Enter number of edges (max 20): 5
Enter edge 1 (u v w): 0 1 2
Enter edge 2 (u v w): 0 2 4
Enter edge 3 (u v w): 1 2 1
Enter edge 4 (u v w): 1 3 7
Enter edge 5 (u v w): 2 3 3
Edges in MST:
1 - 2 : 1
0 - 1 : 2
2 - 3 : 3
Total weight = 6

```

OBSERVATION



Lab Program 9:

Implement Fractional Knapsack using Greedy technique.

CODE

```
#include <stdio.h>
struct Item {
    int weight;
    int value;
    float ratio;
};

// Function to sort items by ratio (descending)
void sortItems(struct Item items[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (items[i].ratio < items[j].ratio) {
                struct Item temp = items[i];
                items[i] = items[j];
                items[j] = temp;
            }
        }
    }
}

float fractionalKnapsack(struct Item items[], int n, int capacity) {
    sortItems(items, n);

    float totalValue = 0.0;

    for (int i = 0; i < n; i++) {
        if (capacity >= items[i].weight) {
            // Take the whole item
            capacity -= items[i].weight;
            totalValue += items[i].value;
        } else {
            // Take fraction of the item
            totalValue += items[i].ratio * capacity;
            break;
        }
    }

    return totalValue;
}

int main() {
    int n, capacity;

    printf("Enter number of items: ");
    scanf("%d", &n);

    struct Item items[n];
```

```

printf("Enter knapsack capacity: ");
scanf("%d", &capacity);

printf("Enter weights and values of items:\n");
for (int i = 0; i < n; i++) {
    printf("Item %d (weight value): ", i + 1);
    scanf("%d %d", &items[i].weight, &items[i].value);
    items[i].ratio = (float)items[i].value / items[i].weight;
}

float maxVal = fractionalKnapsack(items, n, capacity);
printf("Maximum value in Knapsack = %.2f\n", maxVal);

return 0;
}

```

OUTPUT

```

PS C:\Users\chint\Downloads\Knapsack Greedy> & "C:\Users\chint\Downloads\Knapsack Greedy\Knapsack.exe"
Enter number of items: 4
Enter knapsack capacity: 25
Enter weights and values of items:
Item 1 (weight value): 5
15
Item 2 (weight value): 7
20
Item 3 (weight value): 15
25
Item 4 (weight value): 10
10
Maximum value in Knapsack = 56.67

```

OBSERVATION

Date / /
Page

Enter profits:

60

100

120

Enter weights:

10

20

30

Enter max capacity:

50

Total Profit: 240

Item Taken are: 1.00, 1.00, 0.67

MG
13/9.

Lab Program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

CODE

```
#include <stdio.h>

#define MAX_VERTICES 10
#define INF 99999

int minDistance(int dist[], int visited[], int V) {
    int min = INF, min_index = -1;
    for (int v = 0; v < V; v++) {
        if (!visited[v] && dist[v] <= min) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}

int main() {
    int V, src;
    int graph[MAX_VERTICES][MAX_VERTICES];
    int dist[MAX_VERTICES];
    int visited[MAX_VERTICES] = {0};

    printf("Enter number of vertices (max %d): ", MAX_VERTICES);
    scanf("%d", &V);
    if (V <= 0 || V > MAX_VERTICES) return 0;

    printf("Enter adjacency matrix (use %d for no edge):\n", INF);
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    printf("Enter source vertex (0 to %d): ", V - 1);
    scanf("%d", &src);
    if (src < 0 || src >= V) return 0;

    for (int i = 0; i < V; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }
    dist[src] = 0;

    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, visited, V);
        if (u == -1) break;
```

```

    visited[u] = 1;

    for (int v = 0; v < V; v++) {
        if (!visited[v] && graph[u][v] != INF && dist[u] + graph[u][v] < dist[v]) {
            dist[v] = dist[u] + graph[u][v];
        }
    }
}

printf("Shortest distances from vertex %d:\n", src);
for (int i = 0; i < V; i++) {
    if (dist[i] == INF)
        printf("%d: INF\n", i);
    else
        printf("%d: %d\n", i, dist[i]);
}

return 0;
}

```

OUTPUT

```

Enter number of vertices:4
0 10 999 30
10 0 50 999
999 50 0 20
30 999 20 0
Enter source vertex0
dist[0]=0
dist[1]=10
dist[2]=50
dist[3]=30

Process returned 4 (0x4)   execution time : 71.544 s
Press any key to continue.
|

```


OBSERVATION

```

for(int i=0; i<n; i++){
    printf("dist [%d] = %d\n", i, dist[i]);
}
return 0;

```

output

Enter number of vertices: 4

Cost matrix

0	10	999	30
10	0	50	999
999	50	0	20
30	999	20	0

src vertex 0

dist[0] = 0

dist[1] = 10

dist[2] = 50

dist[3] = 30

MS
2/1/21

Lab Program 11:

Implement “N-Queens Problem” using Backtracking.

CODE

```
#include <stdio.h>
#include <stdlib.h>

int place(int x[], int k) {
    for (int i = 1; i < k; i++) {
        if (x[i] == x[k] || abs(x[i] - x[k]) == k - i)
            return 0;
    }
    return 1;
}

int main() {
    int n;
    printf("Enter n: ");
    if (scanf("%d", &n) != 1 || n <= 0) return 0;

    int *x = calloc(n + 1, sizeof(int));
    int k = 1;
    x[k] = 0;

    while (k > 0) {
        x[k]++;
        while (x[k] <= n && !place(x, k))
            x[k]++;
        if (x[k] <= n) {
            if (k == n) {
                for (int i = 1; i <= n; i++)
                    printf("%d ", x[i]);
                printf("\n");
            } else {
                k++;
                x[k] = 0;
            }
        } else {
            k--;
        }
    }

    free(x);
    return 0;
}
```

OUTPUT

```
PS C:\Users\BMSCECSE-L3-65\Desktop\IBM24CS084> cd "c:\Users\BMSCECSE-L3-65\Desktop\IBM24CS084\Nqueens" ; if ($?) { gcc Nqueens.c -o Nqueens } ; if ($?) { .\Nqueens }
Enter n: 4
2 4 1 3
3 1 4 2
```

OBSERVATION

```

while (k > 0) {
    n[k]++; while (n[k] <= n) place(n, k);
    n[k]--;
}
if (n[k] <= n) {
    if (k == n) {
        for (int i = 1; i <= n; i++)
            printf("%d ", n[i]);
        printf("\n");
    } else {
        k++; n[k] = 0;
    }
} else {
    k--;
}
}

free(n); return 0;
}

void main() {
    printf("Enter number of queens: ");
    scanf("%d", &n);
    if (n < 1 || n > 10) {
        printf("Invalid input\n");
        return;
    }
    printf("Enter number of queens: ");
    int a[10][10] = {0};
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            a[i][j] = 1;
        }
    }
    solve(a, 0, n);
}

// Output:
// Enter number of queens: 4
// 2 4 1 3
// 3 1 4 2

```